1.1

I'm not great at word games, so when I play Wordle, I often get stuck after a few guesses because I find it hard to keep track of the words that fit the previous guesses, so I end up randomly guessing and losing my streaks. I want to build a tool that does this filtering for me and suggests the next best guess (a Wordle solver).

1.2

A Java program that uses Swing GUI where I enter each Wordle guess and mark the colors for each letter with grey, yellow, or green. The program has a database of many 5-letter worlds and will filter them out based on my feedback. It will then suggest the best next guess from the remaining words.

2.1

	Urgent	Not Urgent
Important	Do (MVP) - Load a 5 letter word list from a text file - Enter a guess and set each letter's color either grey, yellow, or green - Filter the word list to all the words that match the feedback so far - Suggest the next best guess by letter frequency - Create a basic Swing GUI with 6x5 tile grid, input field, and a suggestion display	Schedule (V2.0+) - Optimize guesses that split clears 50% of guesses at a time - Have a separate answer and allowed guess list for larger inputs - Display a remaining word count, and remaining letters left list - Undoing guesses
Not Important	Delegate (Use a library) - GUI components with Java Swing - Words.txt file using a public-domain word list - File reading using Files.readAllLines	Delete (Cut it) - User accounts and a leaderboard - Multiplayer mode - Tile flip animations

	- Data structures (primarily ArrayList and HashMap) - Collections.sort to help with efficient sorting	

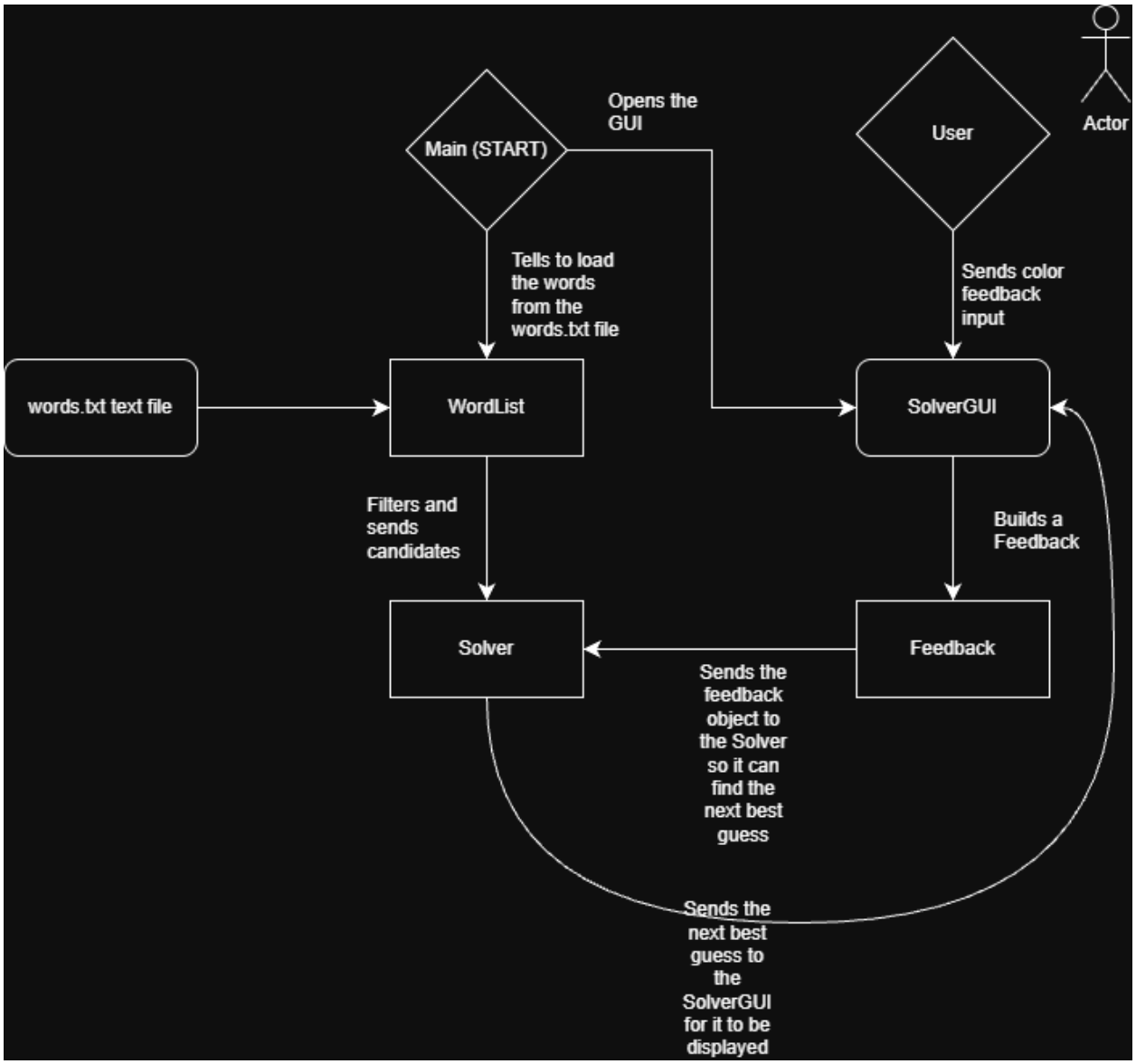
2.2

Java (JDK 25)
Java Swing for the GUI
Java standard libraries like java.util for the List collection. java.nio.file for file reading
A words.txt file from an external source (ENABLE public-domain words list)
Git/GitHub, IntelliJ

2.3

1. Set up the GitHub repo, project structure, README, and words.txt
2. Load the word list into a List<String> and print the word count to confirm it loads
3. Create a Feedback class that represents a guess (gray, yellow, green per letter) and parse it from console input
4. Write the filtering logic that keeps only words consistent with one guess's feedback, and test it against known examples in a main method
5. Make sure that the filters exist after multiple guesses and have a full playable game through the console
6. Add letter-frequency scoring to rank remaining words and suggest the best next guess
7. Create the GUI interface with the 6x5 tile grid, input fields, and suggested word label
8. Connect the GUI to the existing solver. Clicking tiles should change feedback colors and submitting a guess updates the suggestion
9. Handle edge cases like invalid input, no remaining words, and game won

2.4



3.1

https://github.com/Puhalenthi/Jordle-Solver